Objects of type `std::atomic_flag` *must* be initialized with `ATOMIC_FLAG_INIT`. This initializes the flag to a *clear* state. There's no choice in the matter; the flag always starts clear:

```
std::atomic_flag f=ATOMIC_FLAG_INIT;
```

This applies wherever the object is declared and whatever scope it has. It's the only atomic type to require such special treatment for initialization, but it's also the only type guaranteed to be lock-free. If the `std::atomic_flag` object has static storage duration, it's guaranteed to be statically initialized, which means that there are no initialization-order issues; it will always be initialized by the time of the first operation on the flag.

Once you have your flag object initialized, there are only three things you can do with it: destroy it, clear it, or set it and query the previous value. These correspond to the destructor, the `clear()` member function, and the `test_and_set()` member function, respectively. Both the `clear()` and `test_and_set()` member functions can have a memory order specified. `clear()` is a *store* operation and so can't have `memory_order_acquire` or `memory_order_acq_rel` semantics, but `test_and_set()` is a read-modify-write operation and so can have any of the memory-ordering tags applied. As with every atomic operation, the default for both is `memory_order_seq_cst`. For example:

```
f.clear(std::memory_order_release);      <-(1)
bool x=f.test_and_set();                 <-(2)
```

Here, the call to `clear()` (1) explicitly requests that the flag is cleared with release semantics, while the call to `test_and_set()` (2) uses the default memory ordering for setting the flag and retrieving the old value.

You can't copy-construct another `std::atomic_flag` object from the first, and you can't assign one `std::atomic_flag` to another. This isn't something peculiar to `std::atomic_flag` but something common with all the atomic types. All operations on an atomic type are defined as atomic, and assignment and copy-construction involve two objects. A single operation on two distinct objects can't be atomic. In the case of copy-construction or copy-assignment, the value must first be read from one object and then written to the other. These are two separate operations on two separate objects, and the combination can't be atomic. Therefore, these operations aren't permitted.

The limited feature set makes `std::atomic_flag` ideally suited to use as a spin-lock mutex. Initially the flag is clear and the mutex is unlocked. To lock the mutex, loop on `test_and_set()` until the old value is `false`, indicating that *this* thread set the value to `true`. Unlocking the mutex is simply a matter of clearing the flag. Such an implementation is shown in the following listing.

Listing 5.1. Implementation of a spinlock mutex using `std::atomic_flag`

```
class spinlock_mutex
{
    std::atomic_flag flag;
public:
    spinlock_mutex():
```